

Pontificia Universidad Javeriana

Juan David Acuña, Juan Guillermo Gomez Landinez.

Jonathan Martinez, Samuel Bonilla

Taller DevOps: GitHub Actions

1. Introducción

El presente informe corresponde al taller de DevOps, cuyo objetivo fue implementar un flujo de integración y despliegue continuo (CI/CD) utilizando GitHub Actions sobre un proyecto CRUD en java construido con Maven. A lo largo del taller se aplicaron buenas prácticas de control de versiones mediante GitFlow, se generó el empaquetado del proyecto en formato .jar y se diseñaron pipelines automatizados para la validación y despliegue del sistema.

2. Uso de Git, Gestión de Commits y Pull Requests

Durante el desarrollo del taller se aplicó la metodología GitFlow, manteniendo la rama principal develop y una segunda rama llamada database para las tareas específicas. Los commits se realizaron siguiendo el convencional commits, asegurando claridad en los mensajes y trazabilidad de cambios.

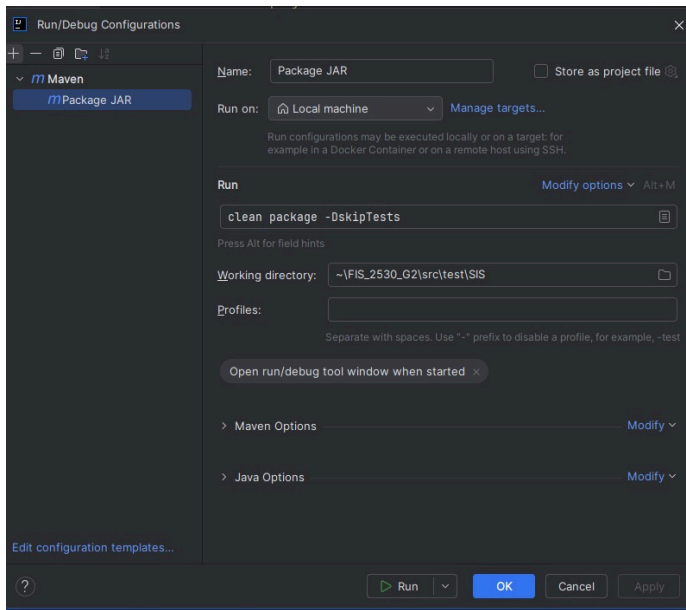
3. Empaquetado del proyecto .jar

Para el desarrollo del proyecto se utilizó un flujo de trabajo basado en ramas, evitando modificar directamente la rama main. Se creó una rama de desarrollo donde se implementaron todas las funcionalidades del sistema, incluyendo controladores, servicios, configuración y pruebas automatizadas.

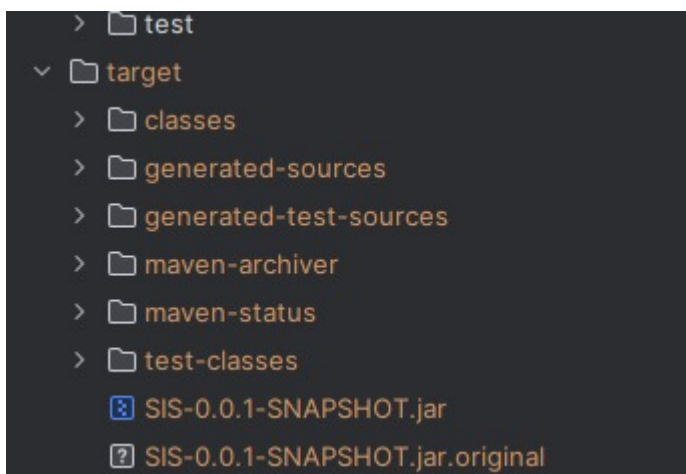
Durante el avance del taller, se realizaron ajustes en archivos clave como pom.xml para configurar dependencias y empaquetado, se añadió la lógica del sistema en el paquete src/main/java y se implementaron pruebas en src/test/java utilizando JUnit y Testcontainers. Esto permitió validar el comportamiento del sistema antes de integrar los cambios.

La estructura del repositorio se mantuvo organizada y se controlaron los cambios de forma progresiva, registrando cada avance por tareas específicas.

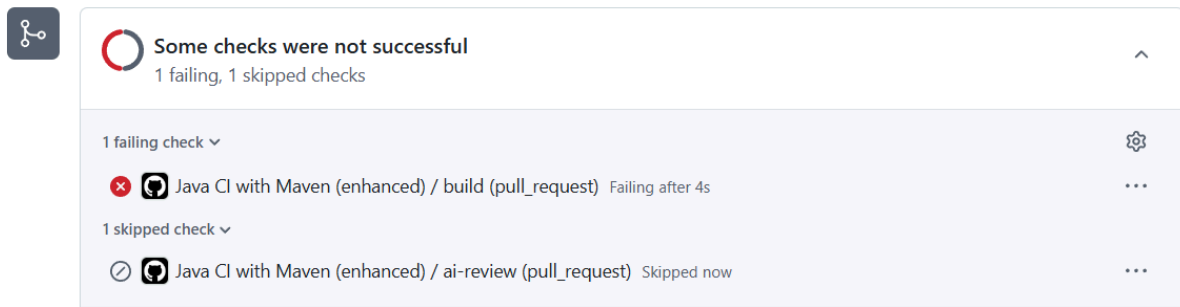
El desarrollo del código se realizó utilizando IntelliJ IDEA, lo cual facilitó la integración con Maven, la ejecución de pruebas, la configuración del empaquetado .jar y la gestión del proyecto en general. El uso de este entorno permitió mantener un flujo de trabajo ordenado, eficiente y alineado con buenas prácticas de control de versiones.



```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <excludes>
      <exclude>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
      </exclude>
    </excludes>
  </configuration>
  <executions>
    <execution>
      <goals>
        <goal>repackage</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```



4. Implementación de GitHub Actions (CI/CD)



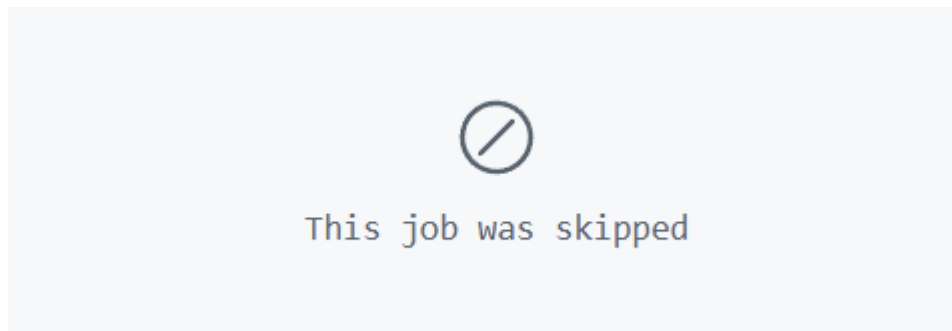
A screenshot of the GitHub Actions summary page. At the top, a red circle icon with a white 'x' is next to the text "Some checks were not successful" and "1 failing, 1 skipped checks". Below this, there are two sections: "1 failing check" and "1 skipped check". The failing check is "Java CI with Maven (enhanced) / build (pull_request)" with a red 'x' icon and the text "Failing after 4s". The skipped check is "Java CI with Maven (enhanced) / ai-review (pull_request)" with a green checkmark icon and the text "Skipped now".

En la captura podemos evidenciar el resultado de ejecución del workflow Java CI with Maven (enhanced) desde github actions.
El pipeline muestra que hubo una prueba fallida en el job build(pull_request) la cual falló 4 segundos después de iniciarse y otra omitida.

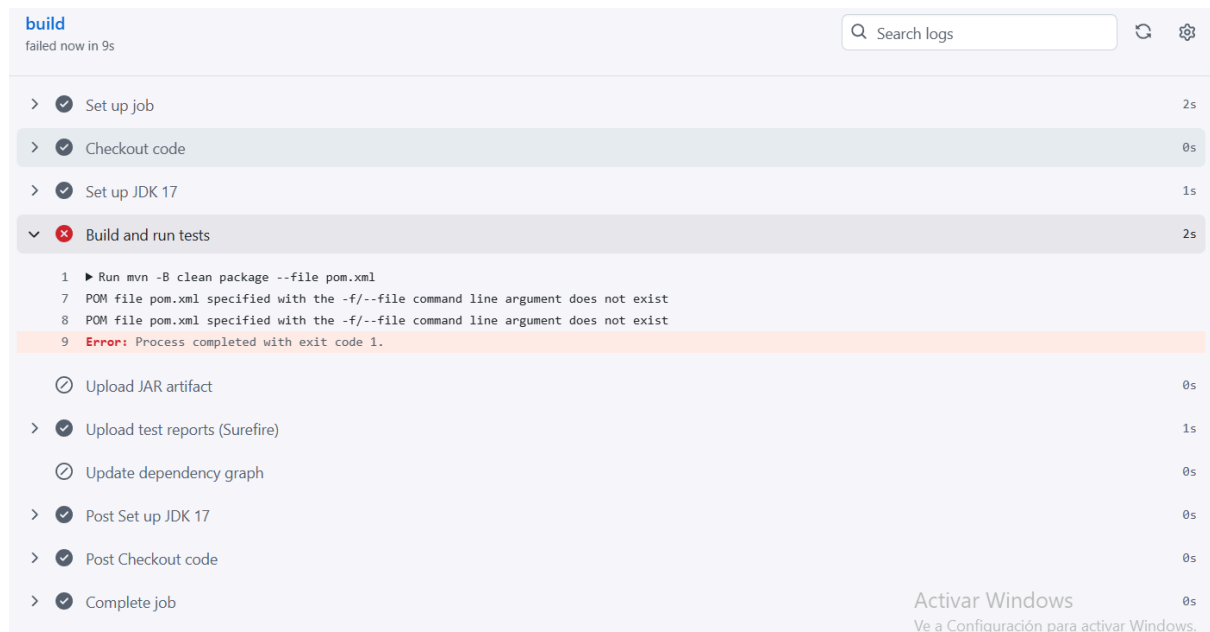


A screenshot of the GitHub Actions logs for the 'build' job. The job is labeled 'build' in blue and 'failed 6 minutes ago in 2s'. There is a search bar labeled 'Search logs'. The logs show a list of steps: 'Set up job', 'Current runner version: '2.328.0'', 'Runner Image Provisioner', 'Operating System', 'Runner Image', 'GITHUB_TOKEN Permissions', 'Secret source: Actions', 'Prepare workflow directory', 'Prepare all required actions', 'Getting action download info', and an error message: 'Error: This request has been automatically failed because it uses a deprecated version of 'actions/upload-artifact: v3'. Learn more: https://github.blog/changelog/2024-04-16-deprecation-notice-v3-of-the-artifact-actions/'.

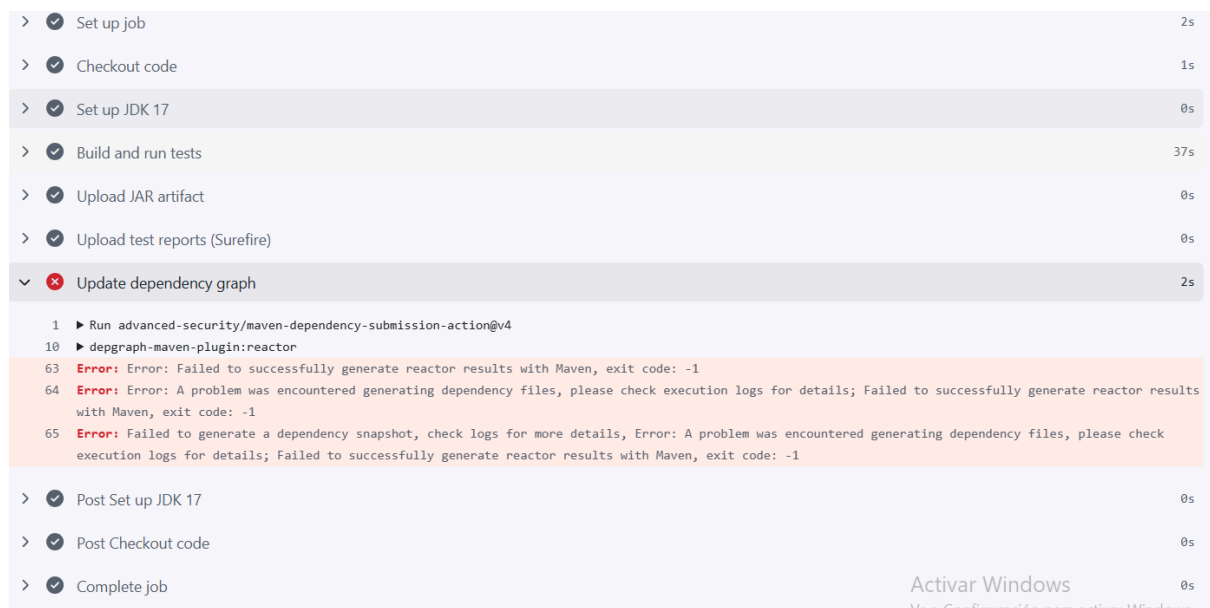
En la captura podemos evidenciar la justificación de porqué falló la ejecución de la primera prueba o check, diciendo que fue automáticamente rechazada debido a que usa una versión antigua del componente “actions/upload-artifact”. Esto hace que cualquier workflow que utilice una versión que no sea la última (v4) falle automáticamente por incompatibilidad.



En la captura se muestra la justificación del chequeo que se saltó.



Como se puede observar en esta imagen, el workflow no encuentra la ruta correcta al archivo POM. Para solucionar esto toca cambiar el directorio de nuestro empaquetado para que funcione acorde a lo que se espera.



Después de solucionar la ubicación del POM y las versiones de las librerías usadas tuvimos este problema.

Gracias a esto, el paso de actualización del grafo de dependencias fue omitido, ya que requiere permisos de GitHub Advanced Security no disponibles en repositorios educativos.

← Java CI with Maven (enhanced)

✓ Update maven.yml #8

Summary

Jobs

✓ build

⌚ ai-review

La siguiente captura muestra la ejecución del workflow Java CI with Maven (enhanced). El workflow se ejecutó correctamente al modificar la versión del artefacto actions/upload-artifact de la v3 a la v4.

Descripción del flujo del trabajo:

name: Java CI with Maven (enhanced)

on:

push:

branches: ["database"]

pull_request:

branches: ["database"]

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Checkout code

uses: actions/checkout@v4

with:

fetch-depth: 0

- name: Set up JDK 17

uses: actions/setup-java@v4

with:

distribution: 'temurin'

java-version: '17'

cache: maven

- name: Build and run tests

run: mvn -B clean package -f src/test/SIS/pom.xml

```
- name: Upload JAR artifact
  if: success()
  uses: actions/upload-artifact@v4
  with:
    name: app-jar
    path: target/*.jar
```

```
- name: Upload test reports (Surefire)
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: surefire-reports
    path: target/surefire-reports/**
```

```
# - name: Update dependency graph
# if: success()
# uses: advanced-security/maven-dependency-submission-action@v4
# o mantén la versión que tenías si esta acción no tiene release v4
```

ai-review:

needs: build

runs-on: ubuntu-latest

if: github.event_name == 'pull_request' && contains(github.head_ref, 'grupo#')

steps:

```
- name: Checkout PR
  uses: actions/checkout@v4
  with:
    fetch-depth: 0
```

```
- name: Generate diff
  run: git --no-pager diff --name-only origin/${{ github.base_ref }}...${{ github.head_ref }} >
changed_files.txt
```

```
- name: IA - Sugerir pruebas a ejecutar (conceptual)
  run: |
    echo "Archivos modificados detectados:"
    cat changed_files.txt
    echo "La IA analizó estos cambios y sugirió ejecutar pruebas relacionadas con los módulos
afectados."
```

El workflow Java CI with Maven (enhanced) se ejecuta automáticamente cada vez que se realiza un push o un pull request sobre la rama database, asegurando que el código que se integra sobre dicha rama siempre sea validado y compilado correctamente antes de fusionarse.

Se tienen dos jobs:

- build: Es el trabajo encargado de empaquetado del proyecto en un .jar
- ai-review: Es el trabajo que le solicitamos a ChatGPT, primero depende del éxito de build, genera un archivo con el listado de archivos modificados llamado changed_files.txt . También

genera un análisis asistido por IA (no implementado, meramente conceptual) que revisa los archivos modificados y sugiere pruebas o verificaciones específicas.

5. Desafíos y soluciones

Durante el desarrollo se presentaron dificultades en la configuración del entorno JDK y la correcta distribución de ramas por parte del pipeline. Estos inconvenientes se solucionaron revisando la documentación oficial de GitHub Actions.

6. Conclusión

El taller permitió comprender y aplicar los fundamentos de CI/CD en un entorno real con GitHub Actions. Se consolidaron conocimientos sobre la automatización del ciclo de vida del software, desde la construcción del código hasta la validación y empaquetado.